In [ ]:
```
Why NumPy is Used?

Normal Python lists are slower for numerical operations.

NumPy provides:

Faster calculations
Less memory usage
Powerful mathematical functions
```

CREATING NUMPY ARRAY

In [14]:
```
import numpy as np
a = np.array([1,2,3,4,5])
print(a)
```

```
[1 2 3 4 5]
```

In [174…
```
#2d
b = np.array([[1,2,3,4],[5,6,7,8]])
print(b)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [13]:
```
#3D
c = np.array([[[1,2,3,5],[1,2,3,4]],[[1,2,3,4],[4,5,6,7]]])
print(c)
```

```
[[[1 2 3 5]
  [1 2 3 4]]

 [[1 2 3 4]
  [4 5 6 7]]]
```

In [20]:
```
#dtype... used to change datatype
a = np.array([1,2,3,4],dtype = float)
a
```

Out[20]:
```
array([1., 2., 3., 4.])
```

In [23]:
```
#arange...print the number from the range provided
a = np.arange(1,20,2)
a
```

Out[23]:
```
array([ 1,  3,  5,  7,  9, 11, 13, 15, 17, 19])
```

In [29]:
```
#reshape...is used to reshape the form
a = np.arange(8).reshape(2,2,2)
a
```

Out[29]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [30]:
```
#np.zeros....prints only zeros  # pass value in a tuple
np.zeros((3,4))
```

Out[30]:
```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

In [31]:
```
#np.ones....prints ones # pass value in a tuple
np.ones((2,4))
```

Out[31]:
```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

In [ ]:
```
#np.random .... prints random number
#  pass value in tuple
np.random.random((3,4))
```

In [38]:
```
#np.linspace....prints the equally spaced numbers between each other
np.linspace(1,10,10,dtype = int)
```

Out[38]:
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [41]:
```
#np.identity....prints the identity matrix
np.identity(3)
```

Out[41]:
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

ARRAY ATTRIBUTES

In [44]:
```
a1 = np.arange(1,11,dtype = int)
a1
```

Out[44]:
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [46]:
```
a2 = np.arange(12,dtype = float).reshape(3,4)
a2
```

Out[46]:
```
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.]])
```

In [48]:
```
a3 = np.arange(8).reshape(2,2,2)
a3
```

Out[48]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [51]:
```
#ndim....prints number of dimensions
a2.ndim
```

Out[51]:
```
2
```

In [53]:
```
#shape....prints the type of shape...likw wheather it is 1d,2d,3d
a3.shape
```

Out[53]:
```
(2, 2, 2)
```

In [55]:
```
#size.....prints the size...no of values present in a array
a2.size
```

Out[55]:
```
12
```

In [58]:
```
#itemsize...works same as size function
a3.itemsize # learn moree abt ittt
```

Out[58]:
```
8
```

In [63]:
```
#dtype ...prints the datatype
a1.dtype
a2.dtype
a3.dtype
```

Out[63]:
```
dtype('int64')
```

CHANGING DATATYPE USING ASTYPE FUNCTION

In [67]:
```
#astype
a3.astype(np.int16)
```

Out[67]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]], dtype=int16)
```

ARRAY OPERATIONS

In [69]:
```
a1 =np.arange(12).reshape(3,4)
a1
```

Out[69]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [71]:
```
a2 = np.arange(12,24).reshape(3,4)
a2
```

Out[71]:
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [79]:
```
# scalar operations....working with one variable
# aritematic operator
a1+3
a1-1
```

```
a1*1
a1 ** 2
```

Out[79]:
```
array([[  0,   1,   4,   9],
       [ 16,  25,  36,  49],
       [ 64,  81, 100, 121]])
```

In [83]:
```
#relational operators
a1 == 3
a1 >= 5
a1 <= 3
a1 != 2
```

Out[83]:
```
array([[ True,  True, False,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True]])
```

In [87]:
```
#vector operations working with 2 variables
# arithematic operators
a1 + a2
a1 - a2
a1 * a2
a1 ** a2
```

Out[87]:
```
array([[                   0,                    1,                 1638
4,
                    14348907],
       [          4294967296,          762939453125,       10155995666841
6,
           113988951853731 43],
       [ 1152921504606846976, -1261475310744950487,  186471204942302412
8,
         6839173302027254275]])
```

In [88]:
```
# relational operators
a1 == a2
```

Out[88]:
```
array([[False, False, False, False],
       [False, False, False, False],
       [False, False, False, False]])
```

ARRAY FUNCTIONS

In [103…
```
#round function rounds to a nearest decimal places
a1 = np.random.random((3,3))
np.round(a1*100)
```

Out[103…
```
array([[43., 82., 52.],
       [34., 11., 12.],
       [28., 47.,  9.]])
```

In [104…
```
#min...prints the minimum value
# 0---> column and 1----> rows
np.max(a1,axis = 0)
```

Out[104…
```
array([0.4299274 , 0.82326328, 0.52322531])
```

In [105…
```
#max...prints the maximum value
```

```
a1.min()
```

Out[105…
```
np.float64(0.09260020391338397)
```

In [107…
```
#sum....add the values
a1.sum(axis = 1)
```

Out[107…
```
array([1.77641599, 0.56579385, 0.84029456])
```

In [108…
```
#prod....multiplies to all the values
a1.prod(axis = 0)
```

Out[108…
```
array([0.04092808, 0.04125985, 0.00562599])
```

In [110…
```
#mean ==== finds the mean
np.mean(a2)
```

Out[110…
```
np.float64(17.5)
```

In [111…
```
#medain....
np.median(a2)
```

Out[111…
```
np.float64(17.5)
```

In [112…
```
#std
np.std(a2)
```

Out[112…
```
np.float64(3.452052529534663)
```

In [113…
```
#var
np.var(a2)
```

Out[113…
```
np.float64(11.916666666666666)
```

In [114…
```
# trignometric functions can be profermed
np.sin(a1)
```

Out[114…
```
array([[0.41680481, 0.73336821, 0.49967653],
       [0.336403  , 0.10637977, 0.11585697],
       [0.27392098, 0.45308845, 0.09246792]])
```

In [116…
```
np.cos(a2)
```

Out[116…
```
array([[ 0.84385396,  0.90744678,  0.13673722, -0.75968791],
       [-0.95765948, -0.27516334,  0.66031671,  0.98870462],
       [ 0.40808206, -0.54772926, -0.99996083, -0.53283302]])
```

In [117…
```
# dot product
a2 = np.arange(12).reshape(3,4)
a3 = np.arange(12,24).reshape(4,3)
np.dot(a2,a3)
```

Out[117…
```
array([[114, 120, 126],
       [378, 400, 422],
       [642, 680, 718]])
```

In [118…
```
#exponents and log
np.exp(a1)
```

Out[118…
```
array([[1.53714592, 2.27792122, 1.68746147],
       [1.40930217, 1.11246853, 1.1231281 ],
       [1.31978335, 1.60035705, 1.09702306]])
```

In [119…
```
#log
np.log(a1)
```

Out[119…
```
array([[-0.84413892, -0.19447923, -0.6477431 ],
       [-1.06974888, -2.23884584, -2.15315063],
       [-1.28205114, -0.75454022, -2.37946394]])
```

In [ ]:
```
#round/floor/ceil
```

In [122…
```
#floor Always goes DOWN to the nearest integer.
a = np.array([2.4,5.9])
np.floor(a)
```

Out[122…
```
array([2., 5.])
```

In [123…
```
#ceil... always goes UP to the nearest integer
a = np.array([2.4,5.9])
np.ceil(a)
```

Out[123…
```
array([3., 6.])
```

INDEXING AND SLICING

In [ ]:
```
a1 = np.arange(10)
a2 = np.arange(12).reshape(3,4)
a3 = np.arange(8).reshape(2,2,2)
```

In [125…
```
a1
```

Out[125…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [127…
```
a1[1:6]
```

Out[127…
```
array([1, 2, 3, 4, 5])
```

In [128…
```
a1[::-1]
```

Out[128…
```
array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

In [129…
```
a2
```

Out[129…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [132…
```
a2[1,2]
```

Out[132…
```
np.int64(6)
```

In [133…
```
a2[::-1]
```

Out[133…
```
array([[ 8,  9, 10, 11],
       [ 4,  5,  6,  7],
       [ 0,  1,  2,  3]])
```

In [134…
```
a3
```

Out[134…
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [135…
```
a3[1,1,0]
```

Out[135…
```
np.int64(6)
```

In [137…
```
a3[1,1,-2]
```

Out[137…
```
np.int64(6)
```

In [138…
```
a3[::-1]
```

Out[138…
```
array([[[4, 5],
        [6, 7]],

       [[0, 1],
        [2, 3]]])
```

ITERATIONS

In [139…
```
for i in a1:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

In [141…
```
for i in a2:
    print(i)
```

```
[0 1 2 3]
[4 5 6 7]
[ 8  9 10 11]
```

In [142…
```
for i in a3:
    print(i)
```

```
[[0 1]
 [2 3]]
[[4 5]
 [6 7]]
```

In [146…
```
for i in np.nditer(a2):
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
```

In [147…
```
for i in np.nditer(a3):
    print(i)
```

```
0
1
2
3
4
5
6
7
```

RESHAPING

In [ ]:
```
#reshape
```

In [150…
```
#transpose....changes rows into columns and columns into rows
a2
```

Out[150…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [151…
```
np.transpose(a2)
```

Out[151…
```
array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])
```

In [154…
```
a4 = np.arange(12).reshape(3,4)
a5 = np.arange(12,24).reshape(3,4)
```

In [156…
```
#hstack === horizontal stacking
np.hstack((a4,a5))
```

Out[156…
```
array([[ 0,  1,  2,  3, 12, 13, 14, 15],
       [ 4,  5,  6,  7, 16, 17, 18, 19],
       [ 8,  9, 10, 11, 20, 21, 22, 23]])
```

In [157…
```
a4
```

Out[157…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [158…
```
a5
```

Out[158…
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [168…
```
#vstack === vertical stacking
np.vstack((a4,a5))
```

Out[168…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [166…
```
#hsplit..horizontal splitting
a2
split = np.hsplit(a2,2)
print(split)
```

```
[array([[0, 1],
       [4, 5],
       [8, 9]]), array([[ 2,  3],
       [ 6,  7],
       [10, 11]])]
```

In [172…
```
#ravel() is a NumPy function used to convert a multi-dimensional array in
np.ravel(a2)
```

Out[172…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

In [173…
```
#vsplit....vertical split
a3
```

Out[173…
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [171…
```
v_split = np.vsplit(a3,2)
print(v_split)
```

```
[array([[[0, 1],
        [2, 3]]]), array([[[4, 5],
        [6, 7]]])]
```

In [ ]: